Assignment 3: Poetry to my ears

Overview

This assignment is due at 11:55 pm on Friday the 29th of May as detailed in Moodle. Submission of this assignment will be via this activity on Ed.

This assignment is worth 15 marks and is worth 15% of your grade. The assignment will be composed in two parts, an individual code submission worth 50% and an individual discussion with the teaching team worth the remaining 50%.

This assignment covers content from weeks 8-11, inclusive, and will, by necessity, cover content from weeks 1 - 7 as well.

Before starting this assignment you must review the slide on Academic Integrity.

Your assignment must be written in the ED platform using only the workspaces provided for the assignment. Code is not to be copied and pasted from sources that are not the given ED workspace.

At any point, you may seek assistance from your TAs in your applied class or via a private post on the forum. Any questions of a general nature may be asked using a public post for the benefit of other students.

Academic Integrity

Plagiarism and collusion are viewed as serious offences. All submitted code will be subjected to a similarity checker, and any submissions determined to be similar to a submission from a current or past student will be investigated further. The outcome of the decision pertaining to plagiarism and/or collusion will be determined by the faculty administration. To ensure compliance with this requirement, be sure to do all your coding in the Ed workspace environment and do not copy and paste any code into the workspace environment.

In cases where plagiarism or collusion has been confirmed, students have been severely penalised, ranging from losing all marks for an assignment, to facing disciplinary action at the Faculty level. Monash has several policies in relation to these offences and it is your responsibility to acquaint yourself with these.

Assessment and Academic Integrity Policies and Procedures (https://www.monash.edu/students/study-success/academic-integrity)

In this assessment, you must not use generative artificial intelligence (GenAI) to generate any materials or content in relation to the assessment task. It is prohibited to use GenAI for reasoning or to gain insight into a task, including for the purpose of translation. You should instead be using a dedicated translation service that will not attempt to interpret the question for you. We consider genAI to be, but not limited to, ChatGPT, Claude, Gemini, DeepSeek and GitHub Copilot.

Generative artificial intelligence also takes the form of code completion systems that are built into some editors. It is up to you to make sure that you are not using any generative artificial intelligence.

You are responsible for all code submitted as a part of your assignment.

Code Quality

Some items we consider when marking code quality:

- Using meaningful variable names

```
def calculate_total(a, b, c):
    x = a + b
    y = x * c
    return y

def calculate_total(price, tax, quantity):
    subtotal = price + tax
    total_cost = subtotal * quantity
    return total_cost
```

Consider the following:

- Clear and meaningful loop and conditional statements
- Meaningful comments
    - Comments should be used to explain the purpose of sections of code **not** to restate what the code in English is.

Good comments:

```
def calculate_discount(price, discount_rate):
    # Ensure discount_rate is treated as a percentage (0–100)
    if discount_rate > 1:
        discount_rate = discount_rate / 100

    # Apply discount
    return price - (price * discount_rate)
```

Over commenting (bad):

```
def calculate_discount(price, discount_rate):
    # Take price
    price = price

    # Check if discount_rate is greater than 1
    if discount_rate > 1:  # if the discount rate is more than 1
        # Divide discount_rate by 100
        discount_rate = discount_rate / 100

    # Multiply price by discount_rate
    discount = price * discount_rate

    # Subtract discount from price
    return price - discount
```

- Maintaining consistent indentation and spacing
- Avoiding unnecessary complexity
    - Complexity in this unit simply refers to is the code straightforward to follow without complications. Some examples of overly complex code would be portions of the code having no real effect.
    - There is another definition of complexity this is not taught or assessed in this unit, you are not assessed on the speed / efficiency of your code so long as it runs without error within a reasonable time period (if it takes too long to run the auto marker will give you an error).

The following is a an over exaggeration but in general we do have the ability to make some deductions should there be room for improvement but given it's the first assignment of an entry level programming unit we are understanding.

```
def greet_user():
    # Ask for name with redundant type conversions
    raw_input_value = input("What is your name? ")
    name_as_string = str(raw_input_value)   # redundant, already str

    # Extra step: copy into another variable
    confirmed_name = "{}".format(name_as_string)  # unnecessary .format

    # Create unused variable
    unused_variable = confirmed_name.upper()

    # Pointless boolean check
    is_valid = True if len(confirmed_name) > 0 else False

    if is_valid:
        # Overly verbose string concatenation
        greeting = "Hello, " + confirmed_name + "!"
        return greeting
    else:
        return "You didn't enter a name."

# Reasonable implementation
def greet_user():
    name = input("What is your name? ")
    if name:
        return f"Hello, {name}!"
    else:
        return "You didn't enter a name."
```

- Structuring your code logically and concisely
- Avoiding magic strings / magic numbers:
  - These are values that have no explanation or context.
- Docstrings should be provided for all functions, the preferred format is:
  - Function description
  - Function parameters with types
  - Function returns with types

An example docstring is provided below:

```
def function_with_pep484_type_annotations(param1: int, param2: str) -> bool:
    """Example function with PEP 484 type annotations.

    Args:
        param1: A short description of the first parameter.
        param2: A short description of the second parameter.

    Returns:
        The return value. True for success, False otherwise.

    """
    pass
```

How will my work be assessed?

For this assignment, students are expected to write tests for their own program. As such we will be providing testing ONLY where we deem it appropriate.

In general, your work will be assessed in two ways.

1. **Functionality:** The functionality of your code will be assessed using a series of test cases run by your TAs. These test cases will not be available to you before submission. You are required to think about how to test the code yourself.
2. **Code quality:** Code quality refers to the the design of your code. This includes, but is not limited to variable names, comments, using "snake" case, human readability, and other categories. For more information on commenting see this post (Not all of these examples will be applicable to you at this time, but you should understand each of the examples by the end of the semester).

Functionality

Functionality will be marked by your TA. You must ensure that your assignment aligns with the functionality described in the specification.

Code quality

Code quality will be marked based on the readability and clarity of your coding solutions. This has nothing to do with functionality. A code base with full functionality can earn no marks for code quality, if the code is poorly written.

File naming and layout

You may create any file that you wish. We will assess your code layout based on whether the structure of your code is reasonable. If your code contains the classes `Snake` `Dog` `Cat`, then containing them in a file called `animals.py` would be reasonable. If we were to write tests for functionality in `animals.py` it would be reasonable to put these tests in a file called `test_animals.py`.

Documentation

Any functions that you have created must have documentation. This is regardless of whether we require the function or you decide to create a new function.

Inline comments and Variable naming

Unlike the documentation, you are expected to use in-line comments throughout your program to

make sure it is clear to the reader. The amount of comments necessary depends on how clear the code is by itself. Therefore, a good habit is to write clear code, so that the amount of in-line comments required is reduced. This will also reduce the number of bugs created.

Comment your code as you write it rather than after reaching a final solution, as it is a lot easier to remember why certain decisions were made and the thought process behind them when the code is fresh in your mind.

Ensure your variable names are meaningful and concise so that your code is understandable at first read. When a reader is going through your program, it should read like a self-explanatory story, using in-line comments to explain the logic behind blocks of code and any additional information that is relevant to the functionality of the code.

Submission Requirements

- Your final submission will be the Python files from every task on the Ed lesson. Do **NOT** change the name of any files that have been provided. You may create as many or as few files as you desire.
- You are allowed to import any library that we have used as part of the course. This includes: abc, copy, csv, numpy, pandas, random, re, sys, and timeutils.

Once you have completed all tasks of the assignment to your satisfaction, clicking the blue `Submit` button in the top right corner of the Ed lesson will officially perform your final submission for the assignment. You are only allowed to make **a single submission** of the assignment so please *double check* that you have **completed all questions** before submitting!

Question 1: Tokenize this!

A small digital poetry studio wants to build a simple poetry generator inspired by a collection of short poems.

Rather than writing poetry from scratch, the system learns patterns from the text by looking at short sequences of tokens and what tends to come next. For example, after seeing a phrase like “the bird”, the model may learn that tokens such as “sings” or “flies” often follow.

By repeating this process across the text, the system builds a simple k-gram model, which is simply a sequence of k consecutive tokens that is used as context to predict the next token.

The goal is not to produce perfect poetry, but to generate text that feels similar to the input.

To keep the project manageable, you will build the system in two stages:

- Question 1: build the k-gram model based on provided instructions.
- Question 2: extend the model in Task 1 using object-oriented programming to generate poetry-like text.

Question 1: Build a k-gram Model

In this task, you are asked to construct a simple k-gram language model from a given text.

Step 1: Reading and pre-processing text

You must implement the following functions, snippets given below:

```
def read_text(file_path):
    pass
```

```
def clean_text(text):
    pass
```

Requirements

- `read_text` reads a text file and returns its content as a string. It must handle cases where the file does not exist using `try...except`. Note that the `sample_text.txt` file contains a collection of short poems, where each poem is identified by a numerical index. A numeric index is a line that contains only digits, such as `1`, `2`, or `15`. Each numbered section should be treated as one poem. An example of a few short poems in the `sample_text.txt` file:

```
1
Stray birds of summer come to my window to sing and fly away.
And yellow leaves of autumn, which have no songs, flutter and
fall there with a sigh.

2
O troupe of little vagrants of the world, leave your footprints
in my words.
```

- If the file does not exist, `read_text` should raise `FileNotFoundError` with a custom error message. If the file is successfully read but contains no text after stripping whitespace, it should raise `ValueError`.
- `clean_text` converts text to lowercase and removes punctuation except commas and full stops. Commas and full stops are kept in the text.

Step 2: Build the k-gram model

You must implement the following function:

```
def build_kgram_model(text, k):
    pass
```

This function should construct a k-gram model from a raw text string.

Inside this function, you should:

- clean the text
- split the text into poems using their numeric indices (e.g. 1, 2, 3, ...)
- treat each numbered section as one poem
- convert each poem into tokens, where commas and full stops are treated as separate tokens
- append `"<END>"` after each poem
- combine all tokens into a single list
- construct k-grams from this list

A token is defined as:

1) a space separated word with punctuation removed,

2) a comma or full stop,

3) the special `"<END>"` token.

A k-gram is defined a tuple of k consecutive tokens, used to predict the next token. You may think of this as a sliding window of size k moving through the list of tokens.

Examples

Given the text:

```
1 The bird sings,
2 The moon shines.
```

After processing, we obtain the following list of tokens:

```
tokens = [
    "the", "bird", "sings", ",", "<END>",
    "the", "moon", "shines", ".", "<END>"
]
```

The new lines in the example list are for visual effect and are not part of the required output.

When we set k = 2, the 2-grams are:

```
[
    (("the", "bird"), "sings"),
    (("bird", "sings"), ","),
    (("sings", ","), "<END>"),
    ((",", "<END>"), "the"),
    (("<END>", "the"), "moon"),
    (("the", "moon"), "shines"),
    (("moon", "shines"), "."),
    (("shines", "."), "<END>")
]
```

Model structure

The function should return a dictionary with the k-gram as the keys and a dictionary containing counts of the following words for each k-gram:

```
{
    ("the", "moon"): {"shines": 1},
    ("moon", "shines"): {".": 1},
    ("shines", "."): {"<END>": 1},
    ("bird", "sings"): {",": 1},
    ("sings", ","): {"<END>": 1}
}
```

Another possible dictionary, from a different text, could contain:

`("the", "moon"): {"is": 2, "shines": 1}`

as a key, value pair. This would mean that the sequence `(“the”, “moon”)` is followed by `“is”` twice - and `“shines”` once.

! Error handling

You must handle invalid input:

- if `k` is not an integer, raise `TypeError`
- if `k <= 0`, raise `ValueError`
- if `text` is not a string, raise `TypeError`
- if the text cannot produce any valid k-grams, raise `ValueError`

Step 3: Query the model

```
def get_next_token_options(kgram, model):
    pass
```

This function should:

- return the dictionary of next words for the given k-gram
- If the k-gram is not in the model, return an empty dictionary

Example

For model:

```
model = {
    ("the", "moon"): {"shines": 2, "is": 1},
    ("moon", "shines"): {".": 2},
    ("shines", "."): {"<END>": 2}
}
```

The output of `get_next_token_options(("the", "moon"), model)` should be:

```
{"shines": 2, "is": 1}
```

Step 4: Informal testing

You should verify your functions using small examples and print statements.

Step 5: Unit Testing for Text Processing

Implement the following tests using the unittest module:

- `test_clean_text_logic`: Verifies the correctness of `clean_text` (e.g., `HELLO! "World": sings,` should become `hello world sings,`).
- `test_build_model_basic`: Verify the model returned by `build_kgram_model` (e.g., a phrase appearing twice should update the count).
- `test_invalid_inputs`: Ensure the appropriate errors are raised for the appropriate inputs.
- `test_file_errors`: Verify that `read_text` correctly handles errors.

Question 2: A generator by any other name.

In this question, you will transform your k-gram model from Question 1 into a robust, object-oriented poetry generator. You will implement a base generator class using **Abstract Base Classes (ABC)**, sub-classes with different token-selection strategies, recursive text generation, and dynamic model updates. Before starting this question, you should copy your solution to question 1 into this workspace, so that you can import its functions and/or variables.

Not all method parameters are explicitly specified. You should determine appropriate parameters based on the required functionality.

Step 1: Implement the Base Class

Create a class called `TextGenerator` to manage the model and raw data.

Constructor

```
def __init__(self, text, k):
    pass
```

Requirements:

- Store the raw input `text` and the k-gram size `k`.
- Build and store the k-gram model by calling your Task 1 functions.
- The constructor should be robust; if Task 1 functions raise an error, it should propagate.

Step 2: Access Next Word Options

Implement the following method to interface with the model:

```
def get_next_options(self, kgram):
    pass
```

- Return the dictionary of possible next tokens and their counts for a given k-gram.
- If the k-gram does not exist in the model, return an empty dictionary.

Step 3: Define the Token-Selection Interface

To ensure proper object-oriented design, `TextGenerator` must be an **Abstract Base Class**. The only strict requirement for this abstract base class is that it must have an interface (method signature that is inherited) for choosing a token `choose_next_token(self, options)`. This interface should be

abstract. You may choose to make other required methods follow this convention if you wish.

Step 4: Generate Text Using Recursion

Implement the text generation logic. **Loop-based generation (for/while) is not allowed for the main process.**

```
def generate(self, start_kgram, max_tokens):
    pass
```

- `max_tokens`: The total count of tokens in the output. It includes the `k` tokens from the `start_kgram`.

Input Validation

Raise `ValueError` if:

- `start_kgram` is not a `tuple` or `len(start_kgram) != self.k`.
- `max_tokens < self.k`.
- `start_kgram` does not exist in the model.

Generation & Formatting Rules

1. **Recursion**: Use a recursive helper (e.g., `_generate_recursive`) to return a **list** of tokens.
2. **Stopping**: Stop when `max_tokens` is reached, no next tokens exist, or `"<END>"` is selected.
3. **Formatting (Crucial)**:
    - **Punctuation**: Remove the space before periods and commas (e.g., `"bird ."` → `"bird."`).
    - **Capitalization**:
        - The first word of the output must be capitalized.
        - Any word immediately following a period and a space (`. `) must be capitalized.
    - **Cleanup**: Ensure `"<END>"` is not present in the final string.

Step 5: Dynamic Model Updates

Extend `TextGenerator` to support updates without creating a new instance.

- **`rebuild_model(...)`**: Rebuild the internal dictionary using current state.
- **`update_k(...)`**: Update `self.k` and rebuild the model.
- **`add_poem(...)`**: Append a new poem to `self.text` (ensure a newline separator) and rebuild the model.

Step 6: Implement Subclasses

Each sub-class below must be implemented as a concrete class that inherits from the `TextGenerator` class, this inheritance can be direct or indirect.

DeterministicGenerator

Override `choose_next_token`:

- Select the token with the highest frequency.
- **Tie-breaking**: Use alphabetical order (e.g., if `"moon"` and `"stars"` both have 2 counts, pick `"moon"`).

RandomGenerator

Override `choose_next_token`:

- Randomly select a token using frequencies as weights (use `random.choices`).

HaikuGenerator

Implement the subclass `HaikuGenerator` which inherits from `RandomGenerator`.

Requirements

- `generate_poem()`: Return a 3-line poem as a single string.
- 5–7–5 Pattern: Generate the first line with 5 tokens, the second with 7, and the third with 5 (approximate counts are acceptable).
- Independent Lines: For each line, pick a new starting `kgram` from the model, uniformly at random, to ensure variety.
- Cleaning: Remove any `<END>` tokens from the generated text. Strip stray punctuation (`.`, `,`) and extra spaces from the start and end of each line.

Example

Given the text file, a Haiku poem can be generated as follows:

```
Heart making the branches down
Knows not that man can lie
Your talk was the woman
```

AcrosticPoemGenerator

Implement the class `AcrosticPoemGenerator`, inheriting from `RandomGenerator`.

Requirements

- An attribute `keyword`
- Implement an appropriate constructor to initialize the parent class and store the keyword attribute.
- `generate_poem()`: Return the complete poem as a string.
- **Acrostic Logic**: The generator must ensure that the first letter of each line spells out the keyword in sequence.
- **Search Strategy**: For each letter in the keyword, search the model's keys for `kgrams` that start with that letter. Randomly select a matching `kgram` as the starting point for that line. If no match is found, return a fallback message.
- **Cleanup & Formatting**: Strip leading/trailing punctuation (`.`, `,`). Generate approximately 6 tokens per line.

Example

Given keyword = "python", the `sample_text.txt` file, and k = 2, an Acrostic poem can be generated as follows:

```
Plucking her petals you do not
Your beauty, my heart is
This moment like the snowy summit
Him in silence
Of time
Never be afraid of me
```

Step 7: Unit Testing

Implement tests using the `unittest` module.

Your tests should cover the following:

- `test_generation_logic`: Verify that the generator can produce text from a valid starting k-gram, respects the maximum token limit, and handles stopping conditions appropriately.
- `test_subclass_behaviour`: Verify that the deterministic and random generators implement their token-selection strategies correctly.
- `test_specialised_poems`: Verify that the haiku and acrostic poem generators return outputs with the expected overall structure.
- `test_formatting_and_cleanup`: Verify that generated text is cleaned and formatted appropriately.
- `test_dynamic_updates`: Verify that rebuilding the model, updating `k`, and adding new poems correctly refresh the generator's internal model.
- `test_invalid_inputs`: Ensure the appropriate errors are raised for invalid constructor arguments, invalid generation inputs, and invalid update inputs.

Marking Criteria

Code Artefact (50%): Each task will be marked against the following criteria.

Discussion (50%): A discussion guide can be found here.

Change log

This is your change log. Hopefully this is small.